

Android XML 中 Material 组件深度解析及替代第三方库全指南

在 Android XML 布局开发中，UI 组件的选择直接影响应用的视觉体验、开发效率及兼容性。Google 官方推出的 Material 组件（MDC）是遵循 Material Design 规范的权威方案，而众多第三方库则在特定场景下提供了更灵活的补充。本文将从核心概念、集成配置、组件详解、替代方案等维度，全面解析相关技术要点。

一、Material 组件（Material Components for Android, MDC）全解析

Material Components for Android（简称 MDC）是 Google 基于 Material Design 3（最新设计语言版本）开发的官方 UI 组件库，旨在替代传统的 Android 原生 UI 组件（如 Button、CardView 等），提供更统一、更具质感的交互体验，同时完美适配 XML 布局开发模式。

1.1 核心定位与设计理念

MDC 的核心定位是“**规范统一的 Material Design 实现载体**”，其设计理念围绕“质感、层次、交互”三大核心：

- 质感**：通过阴影层级、圆角弧度、色彩过渡等细节，模拟真实世界的物理质感，让界面更具亲和力；
- 层次**：利用组件的 elevation（海拔）属性和布局层级，清晰区分内容的主次关系，提升信息传递效率；
- 交互**：内置符合用户习惯的动画效果（如按钮波纹、输入框浮动提示），增强操作反馈，降低用户学习成本。

与原生组件相比，MDC 不仅是“样式增强版”，更是“功能扩展版”，同时解决了原生组件在不同 Android 版本间的兼容性问题（支持 Android 5.0+，即 API 21+）。

1.2 核心优势与适用场景

1.2.1 核心优势

- 官方权威维护**：作为 Google 官方组件库，与 Android 系统版本、AndroidX 同步更新，兼容性和稳定性有绝对保障，避免第三方库“停更”风险；
- 设计规范统一**：严格遵循 Material Design 3 标准，组件间视觉风格一致，无需开发者手动统一样式，降低 UI 设计与开发的沟通成本；

3. **XML 原生适配**：所有组件均支持 XML 直接声明，属性命名符合 Android 开发习惯（如 `app:cornerRadius`、`app:backgroundTint`），无需额外编写代码即可实现复杂样式；
4. **功能全面扩展**：在原生组件基础上增加了大量实用功能，如按钮支持图标与文字组合、输入框支持错误提示与计数、卡片支持边框与点击反馈等；
5. **主题化能力强大**：支持通过 `Theme.Material3` 系列主题统一配置应用的主色、辅助色、字体样式等，实现“一处修改，全局生效”；
6. **无障碍支持完善**：内置无障碍属性（如 `contentDescription` 优化、焦点导航），符合 Android 无障碍开发规范，提升应用覆盖人群。

1.2.2 适用场景

MDC 适用于绝大多数 Android 应用开发场景，尤其适合：

- 追求“原生质感”与系统风格统一的应用（如工具类、办公类应用）；
- 需要快速开发且 UI 设计符合主流规范的项目；
- 对兼容性要求高，需适配多版本 Android 系统的应用；
- 遵循 Material Design 设计规范的产品（如 Google 生态相关应用）。

1.3 完整集成与配置流程

MDC 基于 AndroidX 开发，集成过程需依赖 AndroidX 环境，具体步骤如下：

1.3.1 环境准备

确保项目已迁移至 AndroidX（Android Studio 中可通过 **Refactor** → **Migrate to AndroidX** 完成迁移），且 `build.gradle` 中 `compileSdkVersion` 不低于 33（推荐 34）。

1.3.2 引入依赖

在模块级 `build.gradle`（通常是 `app/build.gradle`）的 `dependencies` 节点中添加 MDC 核心依赖：

Code block

```
1 // MDC 核心组件库（包含所有基础 UI 组件）
2 implementation 'com.google.android.material:material:1.12.0'
3
4 // 可选：若需使用 MDC 中的动画辅助工具
5 implementation 'com.google.android.material:material-animation:1.12.0'
6
7 // 可选：若需使用 MDC 中的颜色工具
8 implementation 'com.google.android.material:material-color:1.12.0'
```

添加后点击 “Sync Now” 同步依赖，建议使用最新版本（可在 [Google Maven 仓库](#) 查看最新版本）。

1.3.3 配置 Material 主题

MDC 组件需要依赖 Material 主题才能正常工作，需在 `res/values/styles.xml` 中设置应用主题为 Theme.Material3 系列主题（Material Design 3 对应的主题），常用主题如下：

Code block

```
1  <!-- 日间模式主题，无 ActionBar -->
2  <style name="AppTheme" parent="Theme.Material3.DayNight.NoActionBar">
3
4      <item name="colorPrimary">@color/primary_color</item>
5
6      <item name="colorPrimaryVariant">@color/primary_variant_color</item>
7
8      <item name="colorSecondary">@color/secondary_color</item>
9
10     <item name="colorError">@color/error_color</item>
11
12     <item name="android:colorBackground">@color/background_color</item>
13
14     <item name="colorOnPrimary">@color/text_on_primary</item>
15
16     <item
17         name="textAppearanceHeadlineLarge">@style/TextAppearance.App.HeadlineLarge</item>
18
19 </style>
20
21 <style name="TextAppearance.App.HeadlineLarge"
22     parent="TextAppearance.Material3.HeadlineLarge">
23     <item name="android:textSize">24sp</item>
24     <item name="android:textWeight">700</item>
25 </style>
```

主题配置完成后，需在 `AndroidManifest.xml` 中指定应用使用该主题：

Code block

```
1  <application
2      android:theme="@style/AppTheme"
3      ...>
4      ...
5  </application>
```

1.4 核心组件 XML 详解与实战

MDC 提供了数十种组件，覆盖从基础控件到复杂布局的全场景需求，以下是开发中最常用的核心组件，包含 XML 布局示例、关键属性解析及使用技巧。

1.4.1 按钮组件：MaterialButton

替代原生 Button，支持圆角、图标、渐变背景、波纹效果等，是 MDC 中最常用的组件之一。

核心属性

属性名	作用	示例值
app:cornerRadius	设置按钮圆角弧度	8dp、16dp（圆形按钮可设为高度的一半）
app:icon	设置按钮左侧图标	@drawable/ic_check、 @mipmap/ic_launcher
app:iconGravity	设置图标位置（左、右、上、下）	start、end、top、bottom
app:backgroundTint	设置按钮背景色（支持单一颜色或渐变）	@color/primary、 @drawable/btn_gradient
app:rippleColor	设置按钮点击时的波纹颜色	@color/white_alpha_50
app:strokeWidth	设置按钮边框宽度	1dp、2dp
app:strokeColor	设置按钮边框颜色	@color/primary
app:enabled	设置按钮是否可用（控制灰度状态）	true、false

XML 实战示例

Code block

```
1  <!-- 1. 基础实心按钮 -->
2  <com.google.android.material.button.MaterialButton
3      android:id="@+id/btn_solid"
4      android:layout_width="wrap_content"
5      android:layout_height="48dp"
6      android:layout_margin="8dp"
7      android:text="实心按钮"
8      android:textSize="16sp"
```

```
9      app:cornerRadius="24dp"
10      app:backgroundTint="@color/primary"
11      app:rippleColor="@color/white_alpha_30"/>
12
13 <!-- 2. 带图标的按钮 -->
14 <com.google.android.material.button.MaterialButton
15     android:id="@+id/btn_with_icon"
16     android:layout_width="wrap_content"
17     android:layout_height="48dp"
18     android:layout_margin="8dp"
19     android:text="带图标按钮"
20     android:textSize="16sp"
21     app:cornerRadius="24dp"
22     app:backgroundTint="@color/secondary"
23     app:icon="@drawable/ic_send"
24     app:iconGravity="end"
25     app:iconPadding="8dp"/>
26
27 <!-- 3. 边框按钮 (无填充色) -->
28 <com.google.android.material.button.MaterialButton
29     android:id="@+id/btn_outline"
30     android:layout_width="wrap_content"
31     android:layout_height="48dp"
32     android:layout_margin="8dp"
33     android:text="边框按钮"
34     android:textSize="16sp"
35     app:cornerRadius="24dp"
36     app:backgroundTint="@android:color/transparent"
37     app:strokeWidth="2dp"
38     app:strokeColor="@color/primary"/>
39
40 <!-- 4. 渐变背景按钮 (需先定义渐变 Drawable) -->
41 <com.google.android.material.button.MaterialButton
42     android:id="@+id/btn_gradient"
43     android:layout_width="wrap_content"
44     android:layout_height="48dp"
45     android:layout_margin="8dp"
46     android:text="渐变按钮"
47     android:textSize="16sp"
48     app:cornerRadius="24dp"
49     app:backgroundTint="@drawable/btn_gradient_bg"/>
50
51
52 <shape xmlns:android="http://schemas.android.com/apk/res/android"
53     android:shape="rectangle">
54     <gradient
55         android:angle="0"
```

```
56         android:endColor="@color/primary"
57         android:startColor="@color/secondary"/>
58     <corners android:radius="24dp"/>
59 </shape>
```

1.4.2 卡片组件：MaterialCardView

替代原生 CardView，支持边框、自定义阴影、圆角、点击反馈等，是展示列表项、内容块的核心组件。

核心属性

属性名	作用	示例值
app:cardCornerRadius	卡片圆角弧度	12dp、16dp
app:cardElevation	卡片阴影高度（值越大阴影越明显）	4dp、8dp
app:cardMaxElevation	卡片最大阴影高度（按压时生效）	8dp、12dp
app:cardStrokeWidth	卡片边框宽度	1dp、2dp
app:cardStrokeColor	卡片边框颜色	@color/grey_200、@color/primary
app:cardBackgroundColor	卡片背景色	@color/white、@color/grey_50
app:cardUseCompatPadding	是否启用兼容padding（解决低版本阴影被裁剪问题）	true

XML 实战示例

Code block

```
1 <com.google.android.material.card.MaterialCardView
2     android:id="@+id/card_item"
3     android:layout_width="match_parent"
4     android:layout_height="wrap_content"
5     android:layout_margin="16dp"
6     app:cardCornerRadius="12dp"
7     app:cardElevation="4dp"
8     app:cardMaxElevation="8dp"
9     app:cardStrokeWidth="1dp"
```

```

10     app:cardStrokeColor="@color/grey_200"
11     app:cardBackgroundColor="@color/white"
12     app:cardUseCompatPadding="true">
13
14
15     <LinearLayout
16         android:layout_width="match_parent"
17         android:layout_height="wrap_content"
18         android:orientation="vertical"
19         android:padding="16dp">
20
21         <TextView
22             android:id="@+id/tv_card_title"
23             android:layout_width="wrap_content"
24             android:layout_height="wrap_content"
25             android:text="卡片标题"
26
27             android:textAppearance="@style/TextAppearance.Material3.TitleLarge"/>
28
29         <TextView
30             android:id="@+id/tv_card_content"
31             android:layout_width="wrap_content"
32             android:layout_height="wrap_content"
33             android:layout_marginTop="8dp"
34             android:text="这是一张 Material 风格的卡片，支持阴影、边框和圆角效果，点
35             击时会有阴影变化的反馈。"
36             android:textAppearance="@style/TextAppearance.Material3.BodyMedium"
37             android:textColor="@color/grey_600"/>
38
39         <com.google.android.material.button.MaterialButton
40             android:id="@+id/btn_card_action"
41             android:layout_width="wrap_content"
42             android:layout_height="wrap_content"
43             android:layout_marginTop="16dp"
44             android:text="查看详情"
45             app:cornerRadius="8dp"
46             app:backgroundTint="@color/primary"/>
47
48     </LinearLayout>
49 </com.google.android.material.card.MaterialCardView>

```

点击反馈优化：若需卡片点击时有阴影变化效果，可在代码中设置卡片为可点击状态，并添加点击事件：

```
1  MaterialCardView cardView = findViewById(R.id.card_item);
2  cardView.setClickable(true);
3  cardView.setFocusable(true);
4  cardView.setOnClickListener(v -> {
5      // 卡片点击逻辑
6  });
```

1.4.3 输入框组件：TextInputLayout + TextInputEditText

替代原生 EditText，解决了原生输入框的“提示语浮动”“错误提示”“输入计数”等痛点，是表单开发的必备组件。其中 TextInputLayout 为容器组件，TextInputEditText 为实际输入组件，二者需配合使用。

核心属性（TextInputLayout）

属性名	作用	示例值
app:hint	输入提示语（会浮动到输入框上方）	请输入手机号、请输入密码
app:errorEnabled	是否启用错误提示功能	true
app:error	错误提示文本（需先启用 errorEnabled）	手机号格式错误、密码长度不足6位
app:counterEnabled	是否启用输入计数功能	true
app:counterMaxLength	输入最大长度（配合 counterEnabled 使用）	11、20
app:counterTextColor	计数文本颜色	@color/grey_500
app:passwordToggleEnabled	是否显示密码明文切换按钮（密码输入时）	true
app:passwordToggleDrawable	自定义密码切换按钮图标	@drawable/ic_password_eye
app:startIconDrawable	输入框左侧图标	@drawable/ic_phone、@drawable/ic_lock

XML 实战示例

Code block

```
1  <LinearLayout
```



```
2     android:layout_width="match_parent"
3     android:layout_height="wrap_content"
4     android:layout_margin="16dp"
5     android:orientation="vertical"
6     android:spacing="16dp">
7
8
9     <com.google.android.material.textfield.TextInputLayout
10         android:id="@+id/til_phone"
11         android:layout_width="match_parent"
12         android:layout_height="wrap_content"
13         app:hint="请输入手机号"
14         app:counterEnabled="true"
15         app:counterMaxLength="11"
16         app:startIconDrawable="@drawable/ic_phone">
17
18         <com.google.android.material.textfield.TextInputEditText
19             android:id="@+id/et_phone"
20             android:layout_width="match_parent"
21             android:layout_height="wrap_content"
22             android:inputType="phone"
23             android:maxLength="11"/>
24
25     </com.google.android.material.textfield.TextInputLayout>
26
27
28     <com.google.android.material.textfield.TextInputLayout
29         android:id="@+id/til_password"
30         android:layout_width="match_parent"
31         android:layout_height="wrap_content"
32         app:hint="请输入密码"
33         app:errorEnabled="true"
34         app:counterEnabled="true"
35         app:counterMaxLength="20"
36         app:passwordToggleEnabled="true"
37         app:startIconDrawable="@drawable/ic_lock">
38
39         <com.google.android.material.textfield.TextInputEditText
40             android:id="@+id/et_password"
41             android:layout_width="match_parent"
42             android:layout_height="wrap_content"
43             android:inputType="textPassword"
44             android:maxLength="20"/>
45
46     </com.google.android.material.textfield.TextInputLayout>
```

```

49     <com.google.android.material.textfield.TextInputLayout
50         android:id="@+id/til_verify_code"
51         android:layout_width="match_parent"
52         android:layout_height="wrap_content"
53         app:hint="请输入验证码"
54         app:errorEnabled="true"
55         app:counterEnabled="true"
56         app:counterMaxLength="6"
57         app:startIconDrawable="@drawable/ic_verify_code">
58
59     <com.google.android.material.textfield.TextInputEditText
60         android:id="@+id/et_verify_code"
61         android:layout_width="match_parent"
62         android:layout_height="wrap_content"
63         android:inputType="number"
64         android:maxLength="6"/>
65
66 </com.google.android.material.textfield.TextInputLayout>
67
68 <com.google.android.material.button.MaterialButton
69     android:id="@+id/btn_login"
70     android:layout_width="match_parent"
71     android:layout_height="48dp"
72     android:text="登录"
73     app:cornerRadius="8dp"
74     app:backgroundTint="@color/primary"/>
75
76 </LinearLayout>

```

代码中设置错误提示：

Code block

```

1  TextInputLayout tilPassword = findViewById(R.id.til_password);
2  TextInputEditText etPassword = findViewById(R.id.et_password);
3  MaterialButton btnLogin = findViewById(R.id.btn_login);
4
5  btnLogin.setOnClickListener(v -> {
6      String password = etPassword.getText().toString().trim();
7      if (password.length() < 6) {
8          // 设置错误提示
9          tilPassword.setError("密码长度不能少于6位");
10     } else {
11         // 清除错误提示
12         tilPassword.setError(null);
13         // 执行登录逻辑
14     }

```

1.4.4 其他高频组件速览

- **MaterialToolbar**: 替代传统 ActionBar, 支持自定义导航图标、菜单、标题样式, 可与 DrawerLayout 配合实现侧滑菜单。

```
<com.google.android.material.appbar.MaterialToolbar  
    android:id="@+id/toolbar"  
    android:layout_width="match_parent"  
    android:layout_height="?attr/actionBarSize"  
    app:title="首页"  
    app:titleTextColor="@color/white"  
    app:navigationIcon="@drawable/ic_menu"  
    app:backgroundTint="@color/primary"/>
```

- **BottomNavigationView**: 底部导航栏，支持图标与文字组合、选中状态切换动画，是应用主导航的常用组件。

```
<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottom_nav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    app:menu="@menu/menu_bottom_nav"
    app:itemIconTint="@drawable/selector_bottom_nav_icon"
    app:itemTextColor="@drawable/selector_bottom_nav_text"
    app:itemBackground="@android:color/white"/>
```

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/menu_home"
        android:icon="@drawable/ic_home"
        android:title="首页"/>
    <item
        android:id="@+id/menu_message"
        android:icon="@drawable/ic_message"
        android:title="消息"/>
    <item
        android:id="@+id/menu_mine"
        android:icon="@drawable/ic_mine"
        android:title="我的"/>
</menu>
```

- **Chip & ChipGroup**: 标签组件，用于展示“热门”“推荐”等标签，支持单选、多选、删除等功能，ChipGroup 用于管理多个 Chip 的布局。

```
<com.google.android.material.chip.ChipGroup  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:layout_margin="16dp"  
    app:chipSpacing="8dp">
```

```
    <com.google.android.material.chip.Chip  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="热门"  
        app:chipBackgroundColor="@color/primary_alpha_10"  
        app:chipTextColor="@color/primary"  
        app:cornerRadius="16dp"/>
```

```
    <com.google.android.material.chip.Chip  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="新品"  
        app:chipBackgroundColor="@color/secondary_alpha_10"  
        app:chipTextColor="@color/secondary"  
        app:cornerRadius="16dp"/>
```

```
    <com.google.android.material.chip.Chip  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="可删除"  
        app:chipBackgroundColor="@color/grey_100"  
        app:chipTextColor="@color/grey_700"  
        app:cornerRadius="16dp"  
        app:closeIconEnabled="true"/>
```

```
</com.google.android.material.chip.ChipGroup>
```

- **Snackbar**: 替代 Toast，支持显示简短消息并附带操作按钮，可滑动关闭，交互性更强。

```
// 基本使用
```

```
Snackbar.make(view, "操作成功", Snackbar.LENGTH_SHORT).show();
```

```
// 带操作按钮的 Snackbar
```

```
Snackbar.make(view, "文件已删除", Snackbar.LENGTH_LONG)
```

```
.setAction("撤销", v -> {
```

```
    // 执行撤销逻辑
```

```
});
```

```
.setActionTextColor(getResources().getColor(R.color.primary))
```

```
.show();
```

1.5 MDC 高级特性：主题定制与适配

1.5.1 动态主题切换

MDC 支持根据系统主题（浅色/深色）自动切换应用主题，只需将主题父类设置为 Theme.Material3.DayNight 系列，即可实现“跟随系统”的主题效果。若需手动控制主题切换，可通过以下方式：

Code block

```
1  // 切换为深色主题
2  AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_YES);
3  // 切换为浅色主题
4  AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_NO);
5  // 跟随系统主题
6  AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_FOLLOW_SYSTEM);
```

1.5.2 组件样式自定义

若 MDC 组件的默认样式无法满足需求，可通过自定义 Style 实现个性化。例如，自定义 MaterialButton 的样式：

Code block

```
1  <!-- 自定义按钮样式 -->
2  <style name="AppMaterialButton" parent="Widget.Material3.Button">
3      <item name="cornerRadius">8dp</item>
4      <item name="backgroundTint">@color/primary</item>
5      <item name="rippleColor">@color/white_alpha_30</item>
6      <item name="android:textSize">16sp</item>
7      <item name="android:paddingStart">24dp</item>
```

```
8         <item name="android:paddingEnd">24dp</item>
9     </style>
10
11
12     <com.google.android.material.button.MaterialButton
13         android:layout_width="wrap_content"
14         android:layout_height="wrap_content"
15         android:text="自定义样式按钮"
16         style="@style/AppMaterialButton"/>
```

二、MDC 替代第三方 UI 库深度解析

尽管 MDC 功能强大且兼容性好，但在某些特定场景下（如需要个性化风格、复杂动画效果、极简 API 等），第三方 UI 库能提供更高效的解决方案。以下推荐的第三方库均支持 XML 布局开发，且各有侧重，可根据项目需求灵活选择。

2.1 全能型 UI 库：XUI

XUI 是一款功能全面的 Android UI 库，支持 Material Design、iOS、微信、QQ 等多种风格，组件覆盖广，API 简洁，且对 XML 布局友好，适合快速开发各类风格的应用。

2.1.1 核心特性

- **多风格支持**：内置 Material Design、iOS、微信、QQ 等多种预设风格，可通过主题快速切换；
- **组件齐全**：涵盖按钮、输入框、列表、弹窗、导航栏、进度条等 60+ 组件，满足全场景需求；
- **XML 高度适配**：所有组件支持 XML 直接声明，属性命名直观，与原生组件使用习惯一致；
- **主题定制灵活**：支持全局主题配置和局部组件样式自定义，可快速匹配产品设计规范；
- **轻量无依赖**：核心库体积小，不依赖其他第三方框架，兼容性好（支持 Android 4.4+）。

2.1.2 集成与配置

1. 在项目级 build.gradle 中添加仓库：

```
allprojects {
    repositories {
        ...
        maven { url "https://jitpack.io" }
    }
}
```

2. 在模块级 build.gradle 中添加依赖：

```
// 核心库（必选）
implementation 'com.github.xuexiangjys:XUI:1.1.9'
// 可选：如果需要使用某些扩展组件（如日历、图表等）
implementation 'com.github.xuexiangjys:XUI:1.1.9:calendar'
implementation 'com.github.xuexiangjys:XUI:1.1.9:chart'
```

3. 配置主题：在 styles.xml 中设置应用主题为 XUI 对应的主题（以 Material Design 风格为例）：

```
<style name="AppTheme"
parent="XUITheme.Material.DayNight.NoActionBar">
    <item name="colorPrimary">@color/primary</item>
    <item name="colorPrimaryDark">@color/primary_dark</item>
    <item name="colorAccent">@color/accent</item>
    <item name="xui_text_style">@style/XUI.TextStyle.Normal</item>
</style>
```

4. 初始化 XUI：在 Application 类的 onCreate 方法中初始化：

```
public class MyApplication extends Application {
    @Override
    public void onCreate() {
        super.onCreate();
        // 初始化 XUI
        XUI.init(this);
        // 开启调试模式（可选）
        XUI.debug(true);
    }
}
```

2.1.3 核心组件 XML 示例

1. 微信风格按钮

Code block

```
1 <com.xuexiang.xui.widget.button.WeChatButton
2     android:layout_width="match_parent"
3     android:layout_height="48dp"
4     android:layout_margin="16dp"
5     android:text="微信风格主按钮"
6     app:wx_button_type="primary"/>
7
```



```
8 <com.xuexiang.xui.widget.button.WeChatButton
9     android:layout_width="match_parent"
10    android:layout_height="48dp"
11    android:layout_margin="16dp"
12    android:text="微信风格次要按钮"
13    app:wx_button_type="secondary"/>
```

2. 带清除按钮的输入框

Code block

```
1 <com.xuexiang.xui.widget.edittext.ClearEditText
2     android:layout_width="match_parent"
3     android:layout_height="wrap_content"
4     android:layout_margin="16dp"
5     android:hint="请输入搜索内容"
6     android:inputType="text"
7     app:cet_clear_icon="@drawable/ic_clear"
8     app:cet_show_clear_icon="true"/>
```

3. 下拉刷新组件

Code block

```
1 <com.xuexiang.xui.widget.refresh.RefreshLayout
2     android:id="@+id/refresh_layout"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent">
5
6     <androidx.recyclerview.widget.RecyclerView
7         android:id="@+id/recycler_view"
8         android:layout_width="match_parent"
9         android:layout_height="match_parent"/>
10
11 </com.xuexiang.xui.widget.refresh.RefreshLayout>
```

代码中设置刷新监听：

Code block

```
1 RefreshLayout refreshLayout = findViewById(R.id.refresh_layout);
2 refreshLayout.setOnRefreshListener(() -> {
3     // 执行刷新逻辑
4     new Handler(Looper.getMainLooper()).postDelayed(() -> {
5         refreshLayout.finishRefresh(true); // 刷新成功
```

```
6      }, 1000);
7  });
8
9  refreshLayout.setOnLoadMoreListener(() -> {
10      // 执行加载更多逻辑
11      new Handler(Looper.getMainLooper()).postDelayed(() -> {
12          refreshLayout.finishLoadMore(true); // 加载成功
13      }, 1000);
14  });
```

2.1.4 适用场景与优势

适用场景：需要快速开发多风格应用、组件需求全面（如电商、社交类应用）、希望简化复杂 UI 实现（如下拉刷新、弹窗）的项目。

核心优势：相比 MDC，XUI 提供了更多“开箱即用”的复杂组件，且支持多风格切换，开发效率更高；相比其他单一风格库，XUI 灵活性更强，可适应不同产品的设计需求。

2.2 简约质感型 UI 库：SmartisanUI

SmartisanUI 是仿锤子手机（Smartisan OS）的 UI 库，以“简约、精致、易用”为核心特点，组件设计简洁大方，交互流畅，适合追求“极简质感”的应用开发。

2.2.1 核心特性

- **风格统一：**严格遵循锤子 OS 的设计语言，组件风格简约精致，视觉一致性强；
- **组件轻量：**聚焦核心组件（按钮、卡片、输入框等），无冗余功能，体积小；
- **交互流畅：**内置细腻的动画效果（如按钮按压反馈、卡片滑动效果），提升用户体验；
- **XML 易用：**组件属性命名简洁，与原生组件使用方式一致，学习成本低。

2.2.2 集成与配置

Code block

```
1  // 在模块级 build.gradle 中添加依赖
2  implementation 'com.smartisanos:smartisan-ui:1.0.0'
```

无需额外配置主题，直接在 XML 中使用组件即可。

2.2.3 核心组件 XML 示例

1. 渐变背景按钮（SButton）

Code block

```
1 <com.smartisanos.ui.widget.SButton
2     android:layout_width="wrap_content"
3     android:layout_height="48dp"
4     android:layout_margin="16dp"
5     android:text="渐变按钮"
6     android:textSize="16sp"
7     app:s_btn_corner_radius="24dp"
8     app:s_btn_gradient_start_color="@color/primary"
9     app:s_btn_gradient_end_color="@color/secondary"
10    app:s_btn_gradient_angle="0"/>
```

2. 精致卡片 (SCardView)

Code block

```
1 <com.smartisanos.ui.widget.SCardView
2     android:layout_width="match_parent"
3     android:layout_height="wrap_content"
4     android:layout_margin="16dp"
5     app:s_card_corner_radius="12dp"
6     app:s_card_shadow_color="@color/black_alpha_10"
7     app:s_card_shadow_size="4dp"
8     app:s_card_background_color="@color/white">
9
10    <TextView
11        android:layout_width="wrap_content"
12        android:layout_height="wrap_content"
13        android:padding="16dp"
14        android:text="锤子风格卡片，阴影细腻，圆角自然"
15        android:textSize="16sp"/>
16
17 </com.smartisanos.ui.widget.SCardView>
```

3. 带清除按钮的输入框 (SEditText)

Code block

```
1 <com.smartisanos.ui.widget.SEditText
2     android:layout_width="match_parent"
3     android:layout_height="wrap_content"
4     android:layout_margin="16dp"
5     android:hint="请输入内容"
6     android:inputType="text"
7     app:s_edit_text_clear_btn_show="true"
8     app:s_edit_text_clear_btn_src="@drawable/ic_clear"
9     app:s_edit_text_corner_radius="8dp"/>
```

2.2.3 适用场景与优势

适用场景：追求简约精致风格的应用（如工具类、阅读类应用）、对 APK 体积有严格要求的项目。

核心优势：风格独特且统一，组件轻量无冗余，动画效果细腻，相比 MDC 更易实现“锤子风格”的视觉体验。

2.3 单一功能强化库：聚焦特定场景需求

这类库不提供完整的 UI 体系，而是针对 MDC 中某类组件的功能不足进行强化，专注于单一场景，具有“轻量、高效”的特点。

2.3.1 圆形头像库：CircleImageView

MDC 中可通过 ShapeableImageView 实现圆形头像，但配置相对繁琐，而 CircleImageView 是专注于圆形/圆角头像的轻量级库，API 极简，是开发中最常用的头像组件库。

核心特性

- 支持圆形、圆角矩形两种样式；
- 支持设置边框宽度和颜色；
- 体积极小（仅几十 KB），无依赖；
- 适配所有 Android 版本，兼容性好。

集成与使用

1. 添加依赖：

```
implementation 'de.hdodenhof:circleimageview:3.1.0'
```

2. XML 布局示例：

<!-- 圆形头像 -->

```
<de.hdodenhof.circleimageview.CircleImageView
    android:id="@+id/civ_avatar"
    android:layout_width="80dp"
    android:layout_height="80dp"
    android:layout_margin="16dp"
    android:src="@drawable/avatar"
    app:civ_border_width="2dp"
    app:civ_border_color="@color/white"
    app:civ_shadow_width="4dp"
    app:civ_shadow_color="@color/black_alpha_10"/>
```

<!-- 圆角矩形头像（需配合 scaleType 使用） -->

```
<de.hdodenhof.circleimageview.CircleImageView
    android:layout_width="80dp"
    android:layout_height="80dp"
    android:layout_margin="16dp"
    android:src="@drawable/avatar"
    android:scaleType="centerCrop"
    app:civ_border_width="2dp"
    app:civ_border_color="@color/primary"
    app:civ_corner_radius="16dp"/>
```

2.3.2 进度条增强库：MaterialishProgress

MDC 中的 ProgressBar 样式相对单一，而 MaterialishProgress 是一款 Material 风格的进度条扩展库，提供了更多样化的进度条样式和动画效果。

核心特性

- 支持圆形、线性两种进度条类型；
- 支持渐变颜色、自定义进度条宽度；
- 支持设置动画速度、indeterminate 模式（无限循环）；
- 与 MDC 风格一致，可无缝集成。

集成与使用

1. 添加依赖：在模块级 build.gradle 中引入依赖，该库托管于 JCenter，需确保项目已配置相关仓库。

```
implementation 'com.pnikosis:materialish-progress:1.7'
```

2. XML 布局示例：支持线性和圆形两种核心样式，可通过属性快速定制外观。 <!-- 1. 线性进度

条 -->

```
<com.pnikosis.materialishprogress.ProgressWheel
    android:id="@+id/progress_linear"
    android:layout_width="match_parent"
    android:layout_height="8dp"
    android:layout_margin="16dp"
    app:matProg_barColor="@color/primary"
    app:matProg_secondaryColor="@color/grey_200"
    app:matProg_progress="60"
    app:matProg_strokeWidth="8dp"/>
```

<!-- 2. 圆形进度条（可用于加载中状态） -->

```
<com.pnikosis.materialishprogress.ProgressWheel
    android:id="@+id/progress_circular"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_gravity="center"
    android:layout_margin="16dp"
    app:matProg_barColor="@color/primary"
    app:matProg_secondaryColor="@color/grey_200"
    app:matProg_indeterminate="true"
    app:matProg_strokeWidth="4dp"/>
```

3. 代码中控制进度：通过 Java 代码动态更新进度值或切换状态，满足交互需求。

```
ProgressWheel linearProgress = findViewById(R.id.progress_linear);
// 设置具体进度值
linearProgress.setProgress(80);
// 重置进度
linearProgress.setProgress(0);

ProgressWheel circularProgress =
findViewById(R.id.progress_circular);
// 切换为确定进度模式
circularProgress.setIndeterminate(false);
// 动态更新进度
circularProgress.setProgress(30);
```

2.3.3 下拉刷新强化库：SmartRefreshLayout

MDC 未提供专门的下拉刷新组件，原生 SwipeRefreshLayout 样式单一，而 SmartRefreshLayout 是一款功能强大的下拉刷新与上拉加载框架，支持自定义头部/尾部动画、多套预设样式，且兼容所有滑动组件（RecyclerView、ListView 等），是国内开发中使用率极高的单一功能库。

核心特性

- **高度可定制**：支持自定义下拉头部、上拉尾部的布局和动画，可完全匹配产品设计风格；
- **样式丰富**：内置 Material、iOS、京东、淘宝等多种预设刷新样式，开箱即用；
- **兼容性强**：完美兼容 RecyclerView、ListView、ScrollView、WebView 等所有可滑动组件；
- **功能全面**：支持下拉刷新、上拉加载、自动刷新、二级刷新、刷新超时等场景；
- **性能优异**：优化了滑动冲突问题，刷新过程流畅，无卡顿现象。

集成与使用

1. 添加依赖：在模块级 build.gradle 中引入核心库和可选的刷新样式库。 // 核心库（必选）

```
implementation 'com.scwang.smart:refresh-layout-kernel:2.0.5'
```

```
// 经典刷新头（可选，如Material风格）
```

```
implementation 'com.scwang.smart:refresh-header-classics:2.0.5'
```

```
// 自定义刷新头（可选，如球体脉冲效果）
```

```
implementation 'com.scwang.smart:refresh-header-ball:2.0.5'
```

2. XML 布局示例：以搭配 RecyclerView 为例，实现 Material 风格的下拉刷新和上拉加载。

```
<com.scwang.smart.refresh.layout.SmartRefreshLayout  
    android:id="@+id/refresh_layout"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">  
  
    <com.scwang.smart.refresh.header.ClassicsHeader  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        app:srlPrimaryColor="@color/primary"  
        app:srlAccentColor="@color/grey_500"/>  
    <androidx.recyclerview.widget.RecyclerView  
        android:id="@+id/recycler_view"  
        android:layout_width="match_parent"  
        android:layout_height="match_parent"  
        android:padding="8dp"/>  
  
    <com.scwang.smart.refresh.footer.ClassicsFooter  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"/>  
  
</com.scwang.smart.refresh.layout.SmartRefreshLayout>
```


3. 代码中设置监听：通过回调接口处理刷新和加载逻辑，控制刷新状态。 SmartRefreshLayout

```
refreshLayout = findViewById(R.id.refresh_layout);
RecyclerView recyclerView = findViewById(R.id.recycler_view);
// 初始化RecyclerView (省略Adapter等配置)

// 下拉刷新监听
refreshLayout.setOnRefreshListener(refreshLayout -> {
    // 模拟网络请求延迟
    new Handler(Looper.getMainLooper()).postDelayed(() -> {
        // 刷新数据逻辑 (如重新请求接口)
        loadData(0); // 0表示第一页
        // 结束刷新
        refreshLayout.finishRefresh(true); // true表示刷新成功
    }, 1500);
});

// 上拉加载监听
refreshLayout.setOnLoadMoreListener(refreshLayout -> {
    new Handler(Looper.getMainLooper()).postDelayed(() -> {
        // 加载更多数据逻辑 (请求下一页)
        loadMoreData(currentPage + 1);
        // 结束加载
        if (hasMoreData) {
            refreshLayout.finishLoadMore(true);
            currentPage++;
        } else {
            // 没有更多数据时，设置为加载结束状态
            refreshLayout.finishLoadMoreWithNoMoreData();
        }
    }, 1500);
});

// 自动触发刷新 (如进入页面时)
refreshLayout.autoRefresh();
```

(注：文档部分内容可能由 AI 生成)